Data Sharing Between Microservices

BYTEBYTEGO
OCT 17, 2024 • PAID

Microservices architecture has become popular for building complex, scalable software systems.

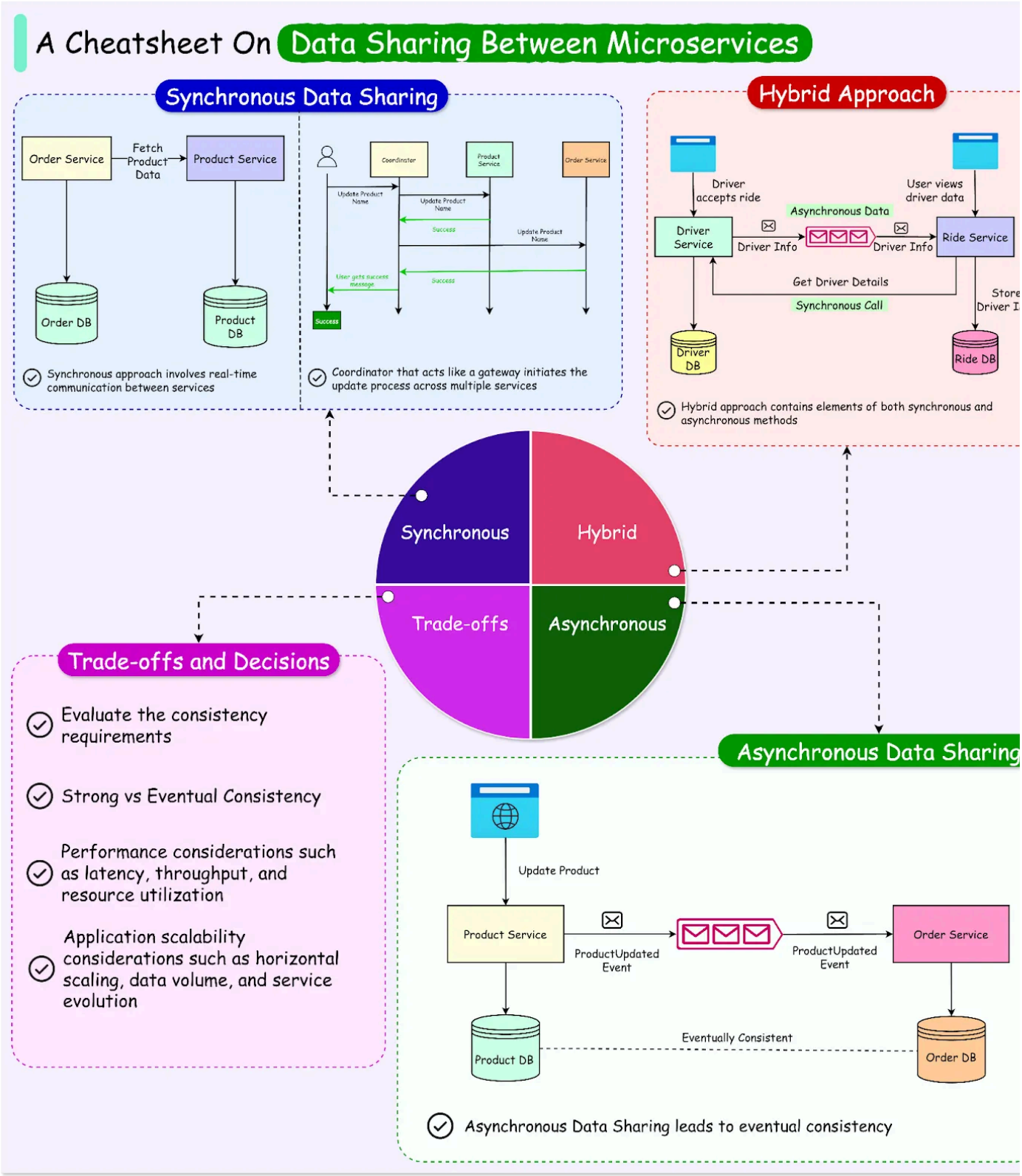
This architectural style structures an application as a collection of loosely coupled, independently deployable services. Each microservice is focused on a specific busir capability and can be developed, deployed, and scaled independently.

While microservices offer numerous benefits, such as improved scalability, flexibili and faster time to market, they also introduce significant challenges in terms of dat management.

One of the fundamental principles of microservices architecture is that each servic should own and manage its data. This principle is often expressed as "don't share databases between services" and it aims to ensure loose coupling and autonomy among services, allowing them to evolve independently.

However, it's crucial to distinguish between sharing a data source and sharing data itself. While sharing a data source (e.g., a database) between services is discouraged sharing data between services is often necessary and acceptable.

In this post, we'll look at different ways of sharing data between microservices and various advantages and disadvantages of specific approaches.



Sharing Data Source vs Sharing Data

As mentioned earlier, it's crucial to understand the difference between sharing a da source and sharing data itself when working with microservices.

This distinction has significant implications for service coupling, independence, an overall system design.

Sharing a data source means multiple services directly access the same database, potentially leading to tight coupling and dependencies. It violates the principle of service data ownership in microservices.

On the other hand, sharing data involves services exchanging data through well-defined APIs or messaging patterns, maintaining their local copies of the data they need. This helps preserve service independence and loose coupling.

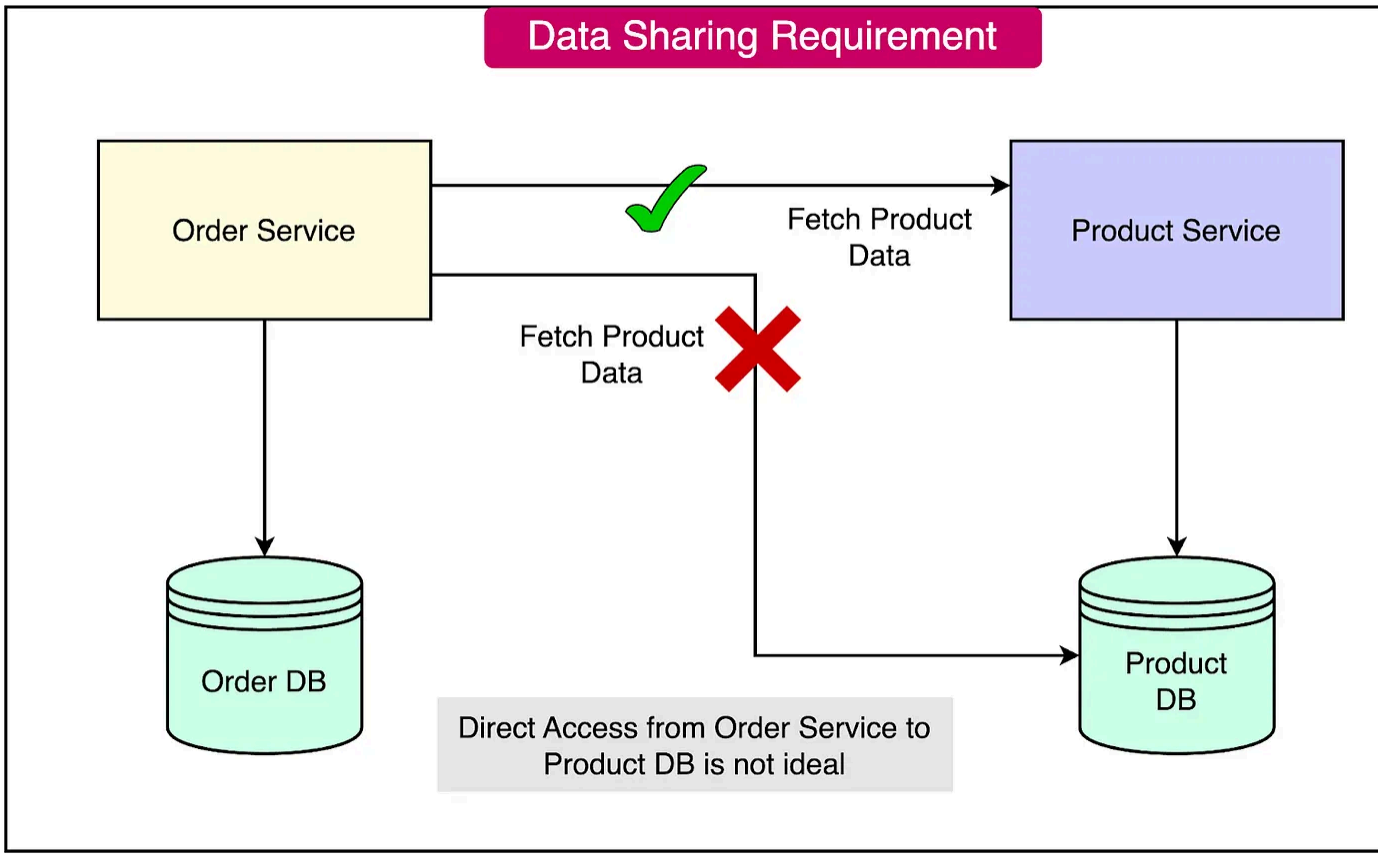
Let's consider an example scenario involving an Order service and a Product servic illustrate the data-sharing dilemma.

- Order Service:
  - Manages order information.
  - Stores order details in its database.
- Product Service:
  - Handles product information.
  - Maintains a separate database for product details.
- Relationship:
  - Each order is associated with a set of products

The challenge arises when the Order service needs to display product information alongside order details.

In a monolithic architecture, this would be straightforward since all the data reside a single database. However, in a microservices architecture, the Order service shoul

not directly access the Product service database. Instead, it should fetch the data vi
the Product Service.



This scenario highlights the need for effective data-sharing strategies in microservices:

- **Data Independence:** Each service owns its data, preventing direct database acc between services.
- **Data Retrieval:** The Order service must find a way to retrieve product informat without violating service boundaries.
- **Performance Considerations:** The chosen data-sharing approach should not significantly impact system performance or introduce excessive latency.
- **Consistency:** Ensuring data consistency between services becomes crucial whe sharing data across service boundaries.

Let us now look at different ways of sharing data.

Synchronous Data Sharing in Microservices

Synchronous data sharing in microservices involves real-time communication betw services, where the requesting service waits for a response before proceeding. This approach ensures immediate consistency but introduces challenges in terms of scalability and performance.

Request/Response Model

The request/response model is the most straightforward synchronous approach:

- Service A sends a request to Service B for data.
- Service B processes the request and sends back a response.
- Service A waits for the response before continuing its operation.

For example, whenever the Order Service is called to provide order details (which includes product information), it has to fetch product data by calling the Product Service.

As expected, the synchronous data-sharing approaches face several challenges:

- **Increased Response Time:** Each synchronous call adds to the overall response time of the system.
- **Cascading Failures:** If one service is slow or down, it can affect the entire chai dependent services.
- **Resource Utilization:** Services may need to keep connections open while waiti for responses, potentially leading to resource exhaustion.
- **Network Congestion:** As the number of inter-service calls increases, network traffic can become a bottleneck.

Due to these challenges, services can share data using duplication. For example, storing the product name with order details within the order table. This way the Or Service does not have to contact the Product Service to fetch the product's name.

However, this creates consistency issues during data updates. For example, the product name is now duplicated across two different services. If the product name changed, the updates have to be done in both the product table and the order table maintain consistency.

Note that this is just a simple example for explanation purposes. Even more critical consistency requirements can exist in a typical application.

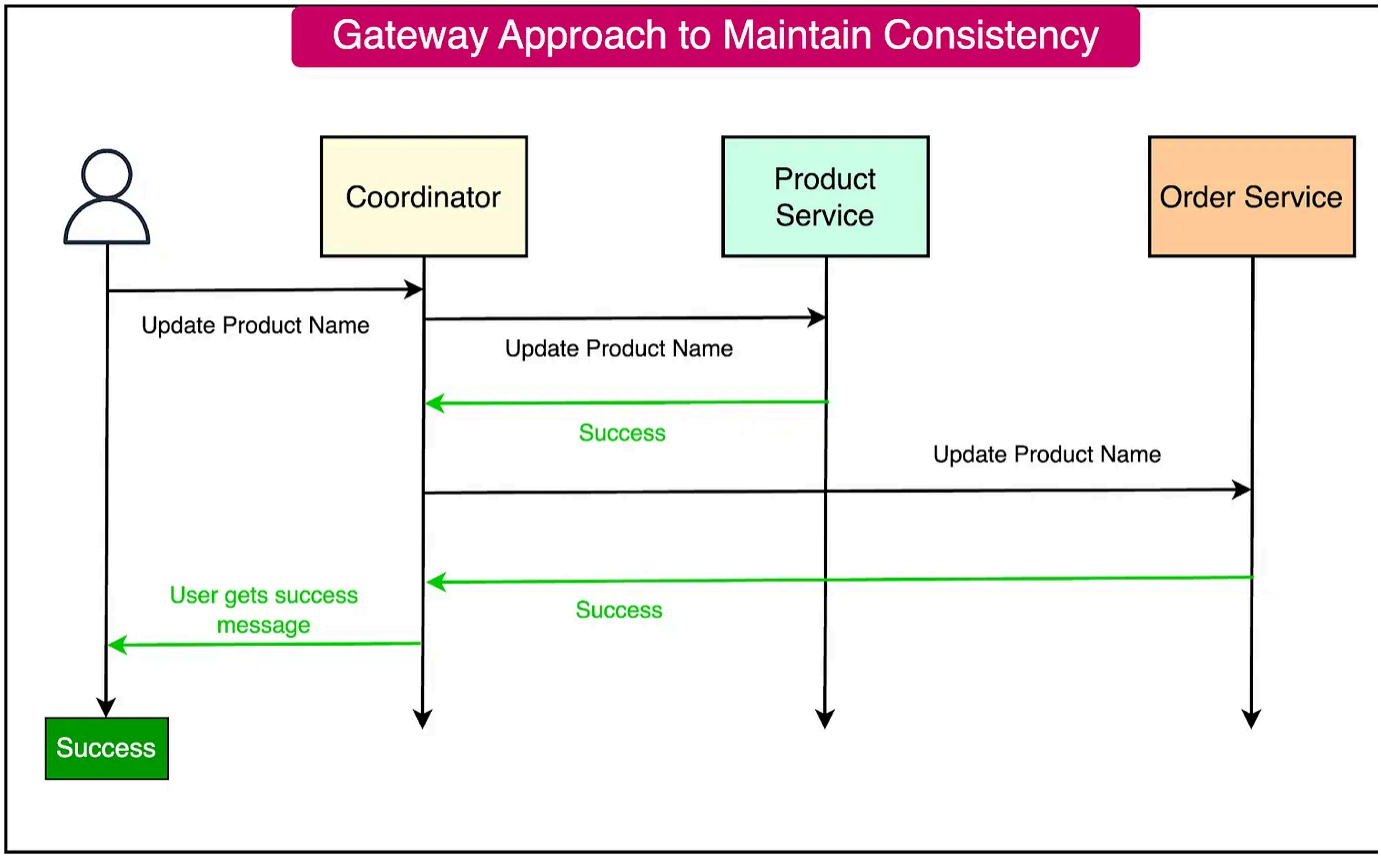
The Gateway Approach

The Gateway approach is a simple approach to ensure data consistency while shari data across multiple services.

Here's an example scenario of how it can work:

- A coordinator that acts like a gateway initiates the update process.
- The coordinator sends update requests to all participating services.
- Each service performs the update and sends back a success or failure response.
- If all services succeed, the transaction is considered complete. If any service fai the coordinator must initiate a rollback on all services.

See the diagram below where a possible change in the product name requires updat in the order service where a copy of the product name is shared. The coordinator ensures that both the updates are successful before informing the user.



Some advantages of the gateway approach are as follows:

- Simpler to implement and reason about
- Maintains consistency

However, there are also disadvantages:

- Less reliable in failure scenarios.
- Poor experience for the users in case one of the services fails.
- Difficult to handle partial failures.

As evident, while synchronous approaches offer strong consistency and simplicity, they come with significant trade-offs in terms of scalability, performance, and resilience.

This is also the reason why asynchronous approaches are usually more popular.

Asynchronous Data Sharing in Microservices

Asynchronous data sharing in microservices architectures enables services to exchange data without waiting for immediate responses. This approach promotes service independence and enhances overall system scalability and resilience.

Let's explore the key components and concepts of asynchronous data sharing.

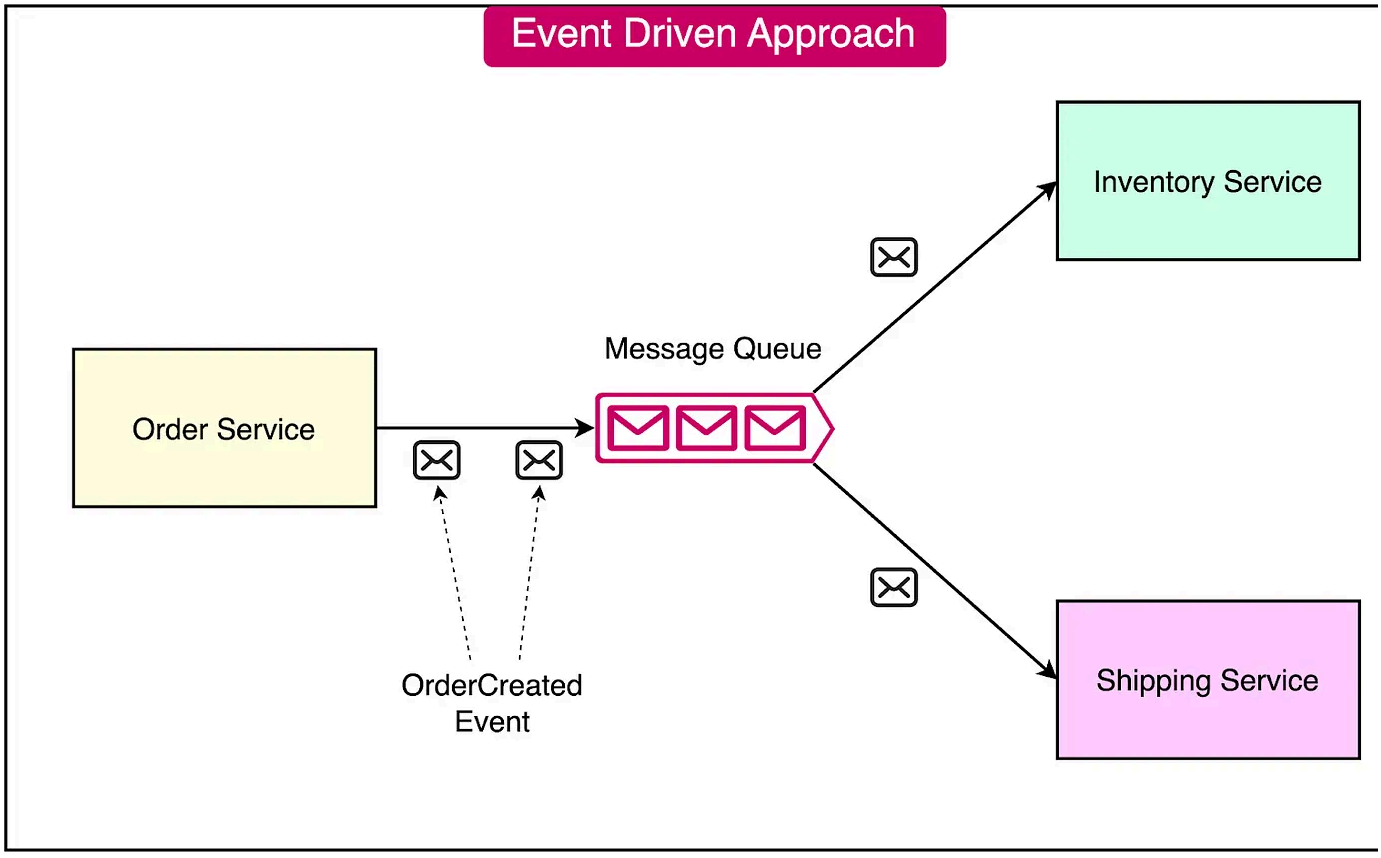
Event-Driven Architecture

In an event-driven architecture, services communicate through events:

- When a service performs an action or experiences a state change, it publishes a event to a message broker.
- Other services interested in that event can subscribe to it and react accordingly
- This loosely coupled approach allows services to evolve independently.

The example below describes such an event-driven approach

- The Order Service publishes an "OrderCreated" event when a new order is plac
- The Inventory Service subscribes to the "OrderCreated" event and updates stoc levels.
- The Shipping Service also subscribes to the event and initiates the shipping process.



Message Queues

Message queues, such as RabbitMQ, serve as the backbone of asynchronous communication in microservices:

- They provide a reliable and scalable way to exchange messages between service
- Services can publish messages to specific topics or queues.
- Other services can consume those messages at their own pace.

Eventual Consistency

Asynchronous data sharing often leads to eventual consistency.

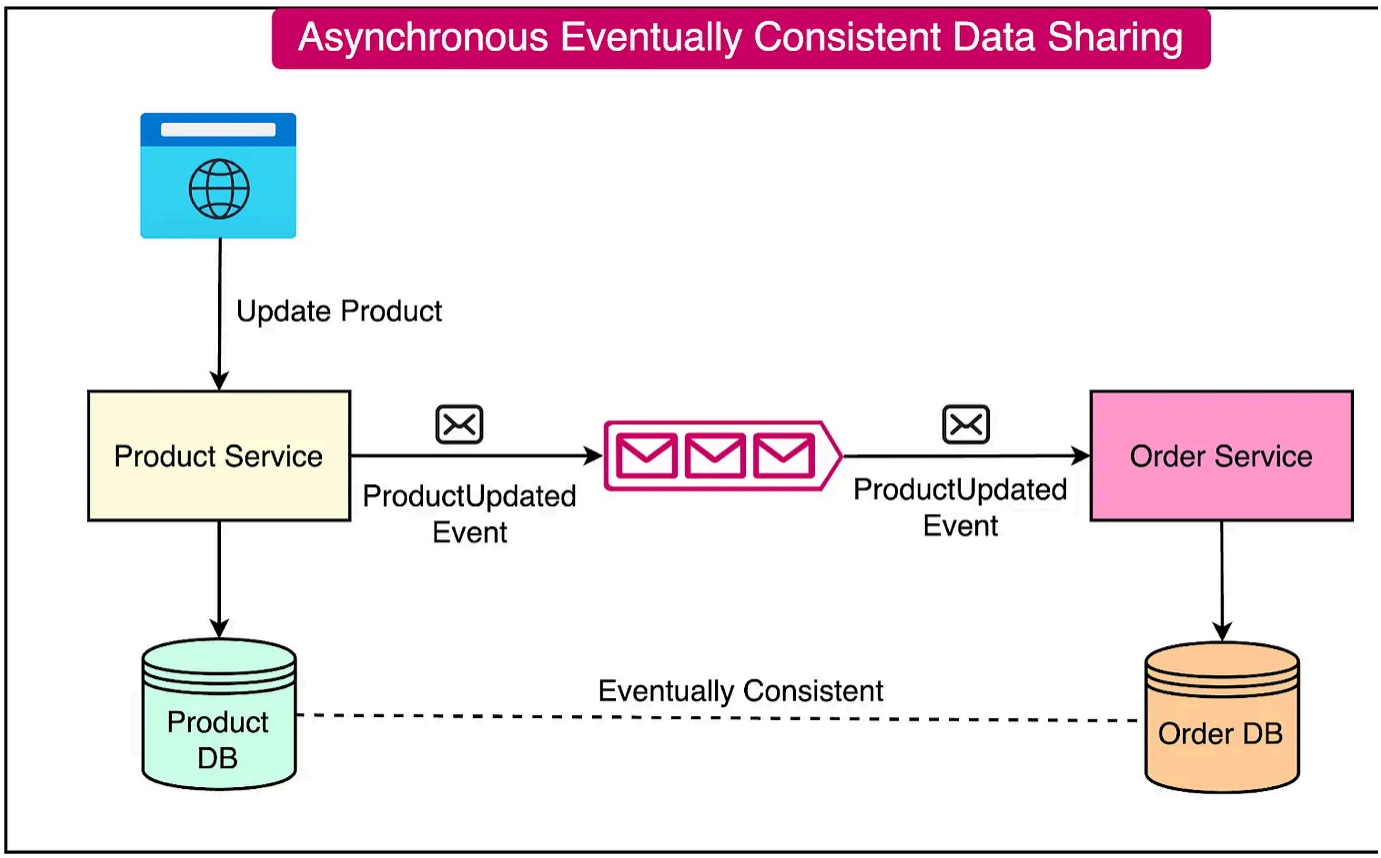
Services maintain their local copies of data and update them based on received ever It means that data across services may be temporarily inconsistent but will eventua reach a consistent state.

This approach allows services to operate independently and improves performance reducing synchronous communication.

Here's an example of the impact of eventual consistency on the earlier example abo data sharing between Product and Order Service:

- The Product Service maintains a local copy of the product data.
- When a product's details (such as name) are updated, the Product Service publishes a "ProductUpdated" event.
- The Order Service consume the event and updates their local copies of the product data asynchronously.
- There may be a short period where product data is inconsistent across services but it will eventually become consistent.

See the diagram below:



The advantages of asynchronous data sharing are as follows:

- **Loose Coupling:** Services can evolve independently without tight dependencies each other.
- **Scalability:** Services can process messages at their own pace, allowing for bette scalability and resource utilization.
- **Resilience:** If a service is temporarily unavailable, messages can be buffered in queue and processed later, improving system resilience.
- **Improved Performance:** Asynchronous communication reduces the need for synchronous requests, resulting in faster response times and improved overall performance.

However, there are also some disadvantages:

- **Eventual Consistency:** Data may be temporarily inconsistent across services, which can be challenging to handle in certain scenarios.
- **Increased Complexity:** Implementing event-driven architectures and handling message queues adds complexity to the system.
- **Message Ordering:** Ensuring the correct order of message processing can be challenging, especially in scenarios where message ordering is critical.
- **Error Handling:** Dealing with failures and errors in asynchronous communicat requires careful design and implementation of retry mechanisms and compensating actions.

Hybrid Approach

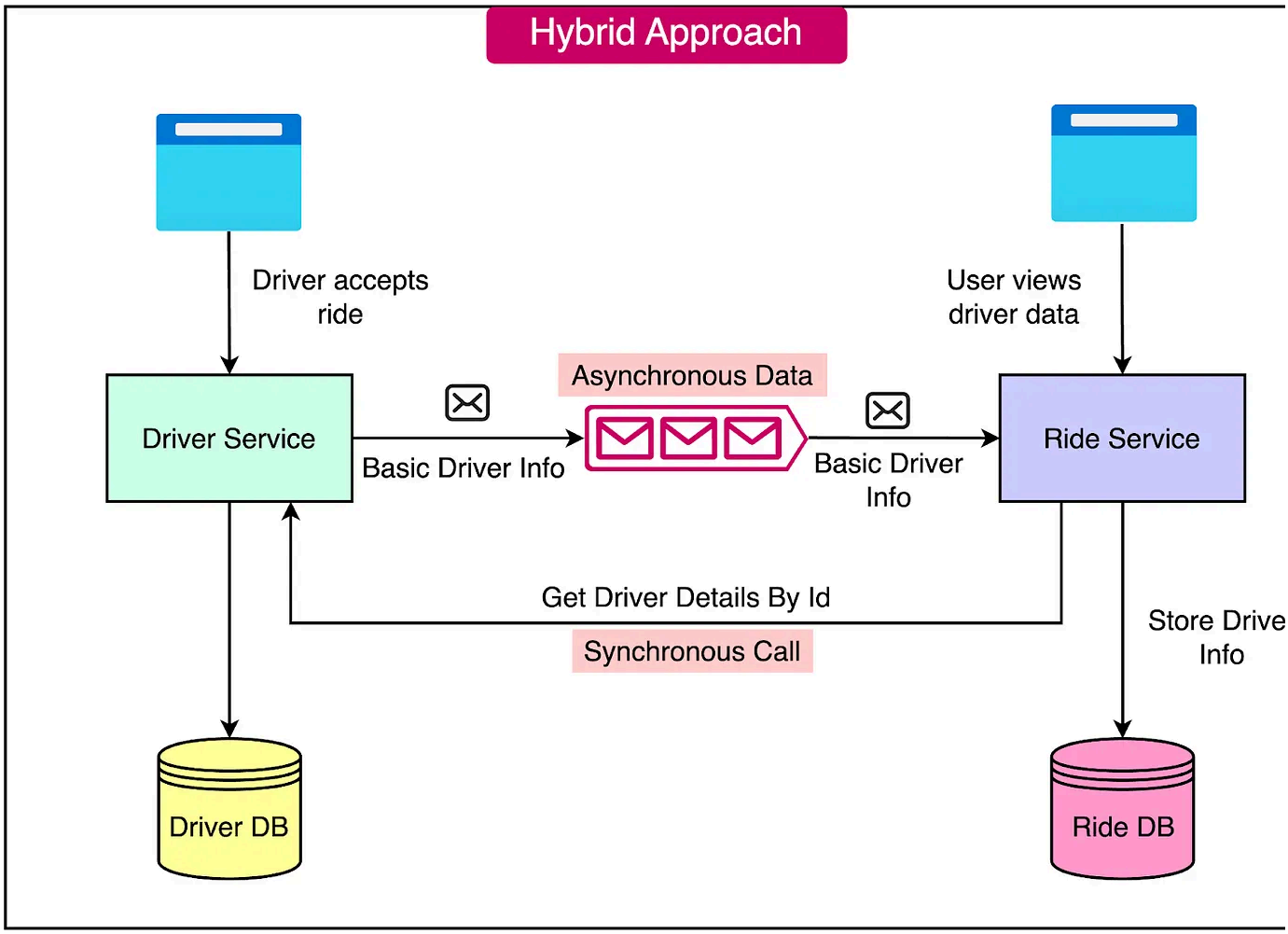
A hybrid approach to data sharing in microservices combines elements of both synchronous and asynchronous methods. This approach aims to balance the benefi of local data availability with the need for up-to-date and detailed information from other services.

Let's consider a ride-sharing application with two microservices: Driver and Ride.

- The Ride service stores basic driver information (ID, name) locally.
- For detailed information (e.g., driver rating, car details), the Ride service makes synchronous call to the Driver service.

This hybrid approach allows the Ride service to have quick access to essential drive data while still retrieving more comprehensive information when needed.

See the diagram below:



The hybrid approach offers a couple of benefits such as:

- **Reduces Redundant Calls:** By storing basic information locally, the Ride servic can reduce the frequency of service-to-service calls. This improves performanc and minimizes the load on the network.
- **Smaller Database Footprint:** Only essential data is duplicated across services, resulting in a smaller database footprint. This saves storage space and reduces overhead of managing redundant data.

However, it also has some drawbacks:

- **Still Affects Response Time:** Although basic information is available locally, retrieving detailed information through synchronous calls to the Driver service still impacts the overall response time of the Ride service.
- **Some Data Duplication:** Even though only essential data is duplicated, there is still some level of data duplication across services. This requires careful management to ensure data consistency and avoid discrepancies.

When implementing a hybrid approach, consider the following:

- **Data Consistency:** Ensure that the duplicated data remains consistent across services. Implement mechanisms to update the local copy of data when change occur in the source service.
- **Caching Strategies:** Employ caching strategies to store frequently accessed detailed information locally, reducing the need for repetitive synchronous calls other services.
- **Asynchronous Updates:** Consider using asynchronous methods, such as event-driven architecture or message queues, to propagate updates to the duplicated data asynchronously, minimizing the impact on response times.

Trade-offs and Decision Making

When designing a microservices architecture that involves data sharing, it's crucial carefully evaluate the trade-offs between consistency, performance, and scalability.

This decision-making process requires a deep understanding of your system's requirements and constraints.

1 - Evaluating Consistency Requirements

One of the key factors in choosing an appropriate solution is the consistency requirement.

Strong Consistency

Strong consistency ensures that all services have the most up-to-date data at all tim It is suitable for systems where data accuracy is critical, such as financial transactic or medical records.

The main characteristics are as follows:

- Often implemented using synchronous communication patterns like the gatew pattern or request-response approach.
- Ensures immediate data consistency across all services.
- Drawbacks include higher latency and reduced availability.

Eventual Consistency

Eventual consistency allows for temporary data inconsistencies but guarantees that data will become consistent over time. It is appropriate for systems that can tolerat temporary inconsistencies, such as social media posts or product reviews.

The primary characteristics of a system based on eventual consistency are as follow

- Implemented using asynchronous communication patterns like event-driven architecture.

- Allows for faster response times and higher availability.
- Requires careful design to handle conflict resolution and compensating transactions.

2 - Performance Considerations

The key performance-related considerations are as follows:

- **Latency**
  - Strong consistency often leads to higher latency due to synchronous communication.
  - Eventual consistency can provide lower latency but may serve st**ale data.**
- **Throughput**
  - Eventual consistency generally allows for higher throughput as services car operate more independently.
  - Strong consistency may limit throughput due to the need for coordination between services.
- **Resource Utilization**
  - Strong consistency may require more computational and network resource maintain data integrity.
  - Eventual consistency can be more efficient in terms of resource usage but requires additional storage for local data copies.

3 - Scalability

The data-sharing solution has a significant impact on the application's scalability. Some of the key aspects are as follows:

- **Horizontal Scaling:** Eventual consistency models often scale better horizontall services can be added without significantly impacting overall system performa

Strong consistency models may face challenges in horizontal scaling due to increased coordination overhead.

- **Data Volume:** As data volume grows, maintaining strong consistency across all services becomes challenging. Eventual consistency can handle large data volu more gracefully but requires careful management of data synchronization.
- **Service Independency:** Eventual consistency allows services to evolve more independently, facilitating easier scaling. Strong consistency models create tigl coupling between services.

Summary

In this article, we've taken a detailed look at the important topic of sharing data between microservices. This is a critical requirement when it comes to building rob and high-performance microservices.

Let's summarize the key learnings:

- The principle of each service owning its data aims to ensure loose coupling anc autonomy among services, allowing them to evolve independently.
- However, it's crucial to distinguish between sharing a data source and sharing data itself.
- Synchronous data sharing in microservices involves real-time communication between services, where the requesting service waits for a response before proceeding.
- Some approaches for synchronous data sharing are the request/response model and the gateway approach
- Asynchronous data sharing in microservices architectures enables services to exchange data without waiting for immediate responses.

- The key components of the asynchronous data-sharing approach are event-driv architecture, message queues, and the concept of eventual consistency.
- A hybrid approach to data sharing in microservices combines elements of both synchronous and asynchronous methods. This approach aims to balance the benefits of local data availability with the need for up-to-date and detailed information from other services.
- Multiple trade-offs need to be taken into consideration while choosing the appropriate data-sharing approach. Some key factors are consistency requirements, performance needs, and the desired scalability of the application

299 Likes · 18 Restacks

Discussion about this post

Comments Restacks

OO Write a comment...